

InstiChain Design Document

We are currently focusing on results of competition conducted by various clubs/associations on campus

Problem and proposed scope

During internship and placement drives, resume verification involves considerable manual work: students must collect and submit proofs, and verifiers must check each document and confirm its authenticity with the issuing organization. Certificates, spreadsheets and emails are maintained independently, so verification becomes difficult when club teams change or the original record keeper is unavailable.

InstiChain will provide faculty-authorized competition records, a student vault for supporting documents, and a verification link for each achievement. A blockchain-inspired append-only ledger will preserve issuance and revocation history, allowing a verifier to check who authorized a record, whether its contents match the issued version, and whether it is still valid.

The first version covers competition results, faculty approval, student accounts, club appointments and proof sharing. Course enrolment, grades and a complete institute directory are future extensions. Uploaded documents alone do not become verified achievements; faculty approval is required.

Users and authority

Role	Permitted actions
Institute administrator	Create clubs; verify and appoint faculty advisors; register advisor public keys; oversee advisor replacement and key recovery.
Faculty advisor	Approve and issue achievements for the assigned club; appoint or revoke club office bearers; revoke or correct that club's issued records.
Club head or office bearer	Create competitions, upload evidence and submit results for approval within the assigned club. Cannot issue achievements or grant roles.
Student	Maintain a private vault, view issued achievements and share selected records for verification.
Verifier	Use a student-provided link to inspect selected claims, issuer authority, current status and permitted proofs.

Account creation and club succession

Verify email ownership through a short-lived, single-use email link or OTP. Accept the domain iitm.ac.in and its subdomains, such as smail.iitm.ac.in, using an exact domain-boundary check; reject lookalike domains. A verified email grants only a basic account, never faculty or club authority. Keep a stable internal user ID separate from the email address.

Each appointment records the club, position, user, grantor, start time and end time. The faculty advisor can revoke it early. A role is active only within its term and before revocation. On annual handover, append revocations or let old terms expire, then append new appointments. Earlier achievements remain valid unless separately revoked. The institute administrator establishes the initial trust in each advisor through an institutional verification process.

Architecture and module split

Use one web application and a modular backend for the prototype. A proposed stack is React for the interface, FastAPI for the API and PostgreSQL for persistent data. Run one authoritative ledger writer; all state-changing requests pass through authentication, club-specific authorization and ledger validation. No mining, cryptocurrency or distributed consensus is required.

Module	Responsibilities and interfaces
Web interface	Student vault, club dashboard, advisor approval screen and verification page; communicates with the backend over HTTPS.
Identity and access	Email verification, sessions and club-scoped permission checks. Recheck active appointments on every protected request.
Clubs and competitions	Club profiles, current office bearers, competition details, result drafts and appointment history.
Approval and credentials	Freeze submitted result versions; validate recipients and proofs; request advisor signatures; issue, revoke and supersede credentials.
Ledger and key registry	Validate signed transactions, order appends, retain advisor key history, create checkpoints and rebuild current-state views.
Vault and storage adapter	Connect cloud storage, upload or select proof files, compute file hashes and enforce document access permissions.
Verification and sharing	Create expiring share links; validate signatures, historical authority, ledger integrity, current revocation state and available proof bytes.

Data and storage boundaries

PostgreSQL stores users, clubs, competitions, result drafts, role grants, credentials, share permissions and the ordered ledger. Current roles and credential statuses are derived views that can be rebuilt by replaying ledger events. Evidence files remain in private cloud storage; the ledger contains opaque IDs, achievement claims and file digests, without access tokens or public document URLs. Email mappings and storage references remain access-controlled outside the ledger.

Core relationships are: a club has competitions and dated role grants; each competition has result submissions; each issued credential names one recipient, an achievement, an advisor and a proof digest. Team results issue one credential per verified member, linked by a common team/result ID. Credentials reference their issuance transaction and any later revocation or replacement.

Vault and Google Drive integration

The vault is a document-management interface, not a cryptocurrency wallet. Connect Google Drive separately from institute email login; the storage account need not use the institute email. Request file-specific OAuth access using drive.file, which supports files created by or explicitly shared with the application [1]. Store refresh tokens encrypted on the server and never embed them in share links.

For issuance, hash the exact final PDF or image bytes and retain that version in a club-managed archive, with a student-visible vault reference or copy. Personal uploads remain unverified until approved. A storage adapter permits another cloud provider later. If a file is deleted, modified or inaccessible, report proof unavailable or hash mismatch separately from the credential's signature and revocation status.

Issuance and verification design

Competition workflow

1. An active club head creates a competition and submits winner details and evidence. The backend resolves recipients to verified accounts, validates the club and freezes a numbered submission version. Unregistered recipients must verify their accounts before issuance.
2. The assigned faculty advisor reviews that exact version and approves or rejects it. Approval covers the recipient IDs, competition, award, proof digests and submission version. Any change to these fields requires a new approval. For bulk results, present the complete recipient list and obtain signatures for each resulting credential.
3. The backend verifies the advisor's signature and current club authority, then atomically appends issuance and updates the credential view. An idempotency key prevents duplicate issuance on retries. Each student sees the credential in the vault and may create an expiring, revocable share link for selected records.
4. A placement verifier opens the link or scans its QR code. The page displays recipient identity, competition, award, approving advisor, issue time and verification results. External verifiers need no institute account, but the unguessable link grants access only to the student-selected records and proofs; it does not expose the entire vault.

Ledger format and cryptographic checks

Represent each committed event as one linked ledger entry containing a schema version, sequence number, transaction ID, event type, object IDs, payload, actor ID, actor key ID, signature, server commit time and previous-entry hash. Event types include ADVISOR_APPOINTED, ROLE_GRANTED, ROLE_REVOKED, ACHIEVEMENT_ISSUED and ACHIEVEMENT_REVOKED. Corrections append a replacement and an explicit supersession reference; existing entries are never edited.

Canonicalize the action payload using JSON Canonicalization Scheme [2], hash proof bytes with SHA-256, and sign the canonical action using Ed25519 [3] from a maintained cryptography library. The ledger writer assigns the sequence and previous hash, then hashes the complete entry excluding its own hash field. Signatures authorize the action; entry hashes bind its place in the ordered history.

For the prototype, advisors sign in the browser using a passphrase-encrypted private key unlocked only for approval. The administrator verifies the advisor-to-public-key binding. Retain old public keys after rotation; record key compromise and recovery explicitly. A compromised key triggers review of affected credentials and explicit revocations, rather than silently replacing history.

Verification checks the payload signature, trusted key binding, advisor authority at the issuance sequence, ledger linkage against a trusted checkpoint, and later revocations or supersessions. If proof bytes are available, recompute their digest. Show separate results for credential authenticity, current status and proof availability, with the time checked. An offline signature check cannot establish that no later revocation exists.

Consistency and trust boundary

Serialize role changes and issuance through the same database transaction and ledger lock, so a racing revocation cannot be bypassed by a stale permission check. Deny ordinary application UPDATE and DELETE privileges on ledger rows. Export signed sequence-and-head-hash checkpoints to a separately controlled institute archive so an auditor can detect rewriting or truncation relative to a retained checkpoint.

This design proves recorded authorization and detects changes relative to trusted keys and checkpoints; it cannot prove that a competition result was truthful. The prototype trusts institute onboarding, advisor judgment and the operating server. A server administrator controlling both data and trust anchors remains a limitation; independent checkpoint retention reduces this risk.

Test plan and acceptance criteria

Area	Tests and expected outcome
Identity and permissions	Accept verified institute domains and subdomains; reject lookalikes, expired/reused OTPs and unverified accounts. Reject student self-promotion, cross-club actions and issuance by club heads.
Role lifecycle	Grant, expire and revoke appointments; test term boundaries and annual handover. A removed head loses access immediately while past achievements remain valid.
Issuance and integrity	Issue a valid individual and team result. Changed recipient, award, proof bytes, payload or signature must fail the relevant check. Editing a submitted version must invalidate its approval.
Ledger and concurrency	Detect modified, reordered and deleted entries against retained checkpoints. Race issuance with advisor revocation; only ledger-ordered authorized actions succeed. Retry approval without duplicate credentials.
Privacy and storage	Deny another student's vault access and expired/revoked share links. Simulate Drive denial, deletion, file replacement and disconnected OAuth; never label missing evidence as verified bytes.
Recovery and end to end	Restore a backup and replay the ledger to the same state. Test key rotation and credential revocation. Demonstrate club submission through advisor approval to placement verification.

Use unit tests for domain checks, role rules, canonicalization and signatures; database integration tests for atomic writes and replay; and browser tests for the full workflow. Proposed load target: 10,000 seeded credentials and 50 concurrent verification requests, with 95% completing within two seconds excluding cloud-file downloads. Record the test environment and actual results.

Revised milestone plan

Proposed six-week schedule from implementation start. This revision prioritizes competition verification and club succession; institute-wide academic records are deferred. Each milestone has a demonstrable completion gate.

Week	Deliverable	Completion gate
1	Scope, schema and account flow	Agree role matrix and threat model; verified login works; seed one club and a trusted advisor.
2	Club roles and competitions	Advisor grants/revokes dated roles; heads submit versioned results; authorization tests pass.
3	Signed ledger and issuance	Advisor signs credentials; atomic append, idempotency, tamper checks and history replay work.
4	Vault and cloud proofs	Real Drive connection, proof hashing and retained copies work; unavailable files are handled clearly.
5	Verification and lifecycle	Share links/QR codes, live revocation checks, corrections and annual handover work end to end.
6	Validation and final demonstration	Run security, concurrency, recovery and load tests; fix failures; deliver setup guide and demonstration.

The critical path is authority setup → role checks → signed issuance → verification. Set up the Drive test account and email delivery in Week 1 to expose access dependencies early. If time is constrained, defer bulk import, notifications and UI polish; retain faculty approval, revocation, proof hashing and access control. Final acceptance is a complete competition-to-resume-verification demo with tampering and team-handover cases.